

- 1 Rstudio
- 2 Working directories (wd)
- 3 Reporting con quarto
- 4 Elementi base
- 5 Strutture
- 6 Funzioni utili
- 7 Monte Carlo
- 8 Esercizi liberi

```

1 # DATA PREPARATION TOL
2 # OTTAVIA, MAGGIO 2023
3
4 rm(list = ls())
5 library(tubridate)
6 library(dplyr)
7
8 setwd("H:/shortcut-targets-by-id/1740KGGXve6z7MtNn00cT3W23DXImV/
9
10 name.data.43 = paste0("AdapTo1_43/", # do not change
11                       "tol43_2023_05_08.csv") # change according t
12 name.data.52 = paste0("AdapTo1_52/", # do not change
13                       "tol52_2023_05_08.csv") # change according t
14 name.data.45 = paste0("AdapTo1_45/", # do not change
15                       "tol45_2023_05_08.csv") # change according t
16 name.env = ls()
17 name.env = name.env[grepl("name.data", name.env)]
18
19 for (i in 1:length(name.env)) {
20   assign(gsub("name.", "", name.env[i]),
21         read.csv(get(name.env[i]), header = T, sep = ","))
22 }
23
1984 total_time
  
```

SCRIPT

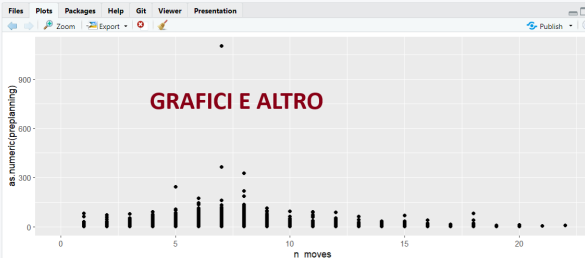
```

R 4.2.3 - H:/shortcut-targets-by-id/1740KGGXve6z7MtNn00cT3W23DXImV/
> ggplot(all.data, aes(x = n_moves, y = as.numeric(preplanning))) + geom_
point()
warning message:
Removed 8 rows containing missing values (geom_point).
>
  
```

CONSOLE/SCRIVANIA

Environment	History	Connections	Tutorial
R - Global Environment	Import Dataset	242 MB	
total_time43	378 obs. of 29 variables		
total_time45	166 obs. of 44 variables		
total_time52	412 obs. of 36 variables		
wide.acc	412 obs. of 28 variables		
Values			
alleged.trials	num [1:3] 20 35 27		
i	12L		
name_desc	chr [1:3] "sbj_char43" "sbj_char45" "sbj_char52"		
name.data.43	"AdapTo1_43/tol43_2023_05_08.csv"		
name.data.45	"AdapTo1_45/tol45_2023_05_08.csv"		
name.data.52	"AdapTo1_52/tol52_2023_05_08.csv"		
name.env	chr [1:3] "data.43" "data.45" "data.52"		
names.print	chr [1:12] "accuracy43" "accuracy45" "accuracy52" "execution43" "execution45" ...		

AMBIENTE



GRAFICI E ALTRO

Ambiente/Environment. Contiene tutti gli oggetti che sono stati creati fino a quel momento nell'ambiente di lavoro

Script. File di tipo testuale dove salvare il codice usato per generare gli oggetti presenti nell'ambiente. Si possono combinare codice e commenti (righe che non vengono eseguite e interpretate come codice #)

```
# codice per assegnare il numero 25 alla variabile x
x <- 25
```

Working directory. La cartella dentro cui sta lavorando R in quel momento. Diventa di fondamentale importanza perché è dove R si aspetta che ci siano i file (e.g., i dati) con cui volete lavorare

Console

I comandi nella console vengono eseguiti con click su Invio

Il codice non viene salvato e scompare sequenzialmente.

L'output appare immediatamente nella console.

Per fare lo switch alla console $\rightarrow \text{ctrl} + 2$

Script

Si possono salvare gli script di codice

Per far “correre” qualsiasi comando nello script → Ctrl + Invio (o cmd + Enter)

L'output appare nella console.

Per fare lo switch allo script $\rightarrow \text{ctr} + 1$

- 1 Rstudio
- 2 Working directories (wd)
- 3 Reporting con quarto
- 4 Elementi base
- 5 Strutture
- 6 Funzioni utili
- 7 Monte Carlo
- 8 Esercizi liberi

- 1 Rstudio
- 2 Working directories (wd)
 - Hard way
 - Easy Way: R Project
- 3 Reporting con quarto
- 4 Elementi base
- 5 Strutture
- 6 Funzioni utili
- 7 Monte Carlo

```
# per sapere dove sta lavorando R
getwd()
```

```
[1] "C:/Users/Ottavia/Documents/GitHub/stpc-analisi-dati/slides/r
```

La wd si può cambiare con il comando `setwd()`:

```
setwd("C:/Users/Ottavia/Documents/altro-percorso")
```


- 1 Rstudio
- 2 Working directories (wd)
 - Hard way
 - Easy Way: R Project
- 3 Reporting con quarto
- 4 Elementi base
- 5 Strutture
- 6 Funzioni utili
- 7 Monte Carlo

Crea automaticamente una working directory in cui potete trovare tutto il materiale necessario per delle analisi

Struttura tipica

```
my-project/  
  
  index.qmd  
  data/  
    mydata.csv  
  img/  
    logo.png  
    plot.jpg  
    illustration.svg  
  bibliography/  
    references.bib  
  slides/  
    intro.qmd  
    advanced-topics.qmd
```

From File → New Project:

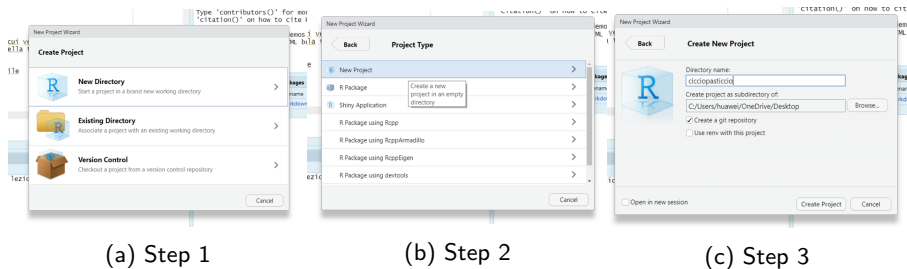


Figure 1: Creare un progetto R in 3 step

Easy Way: R Project

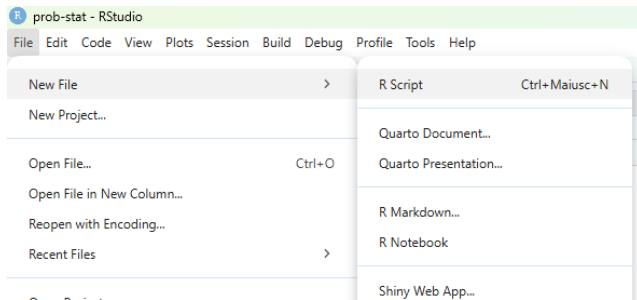


Figure 2: Creare un nuovo script

Oppure la sequenza di tasti: `Ctrl + shift + n`

Per salvare uno script: Click sull'icona del salvataggio

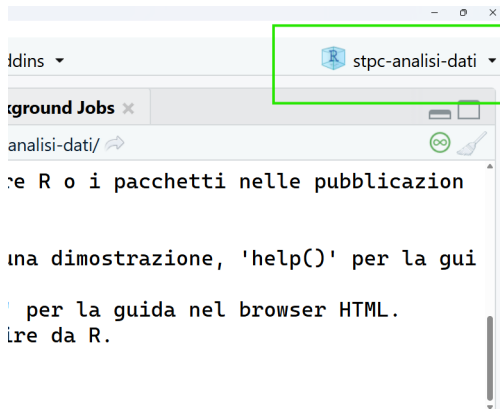


Figure 3: Indica il nome del progetto

- 1 Rstudio
- 2 Working directories (wd)
- 3 Reporting con quarto**
- 4 Elementi base
- 5 Strutture
- 6 Funzioni utili
- 7 Monte Carlo
- 8 Esercizi liberi

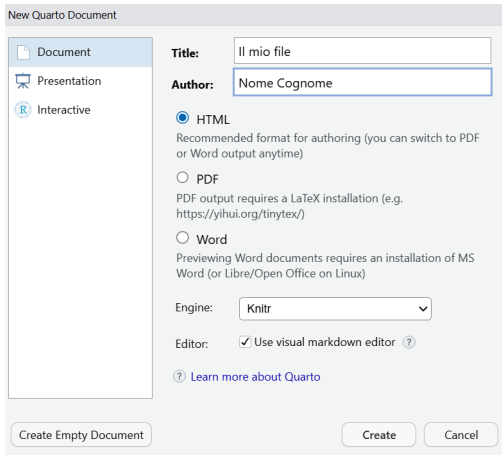
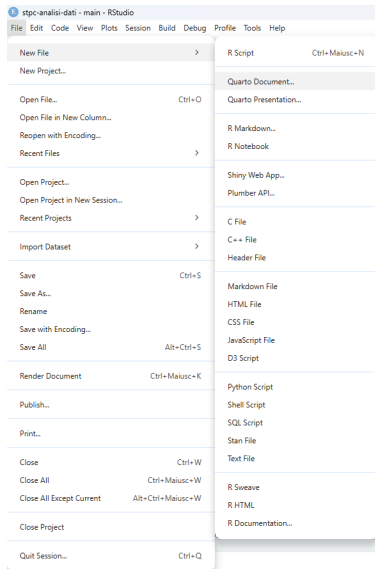
quarto è un markup language integarte con RStudio.

Permette la creazione di documenti (sia pdf sia html) che possono contenere chunk di codice, ovvero delle parti dove viene eseguito il codice R, per ottenere risultati, tabelle e grafici.

In questo corso, useremo quarto per tutti gli esercizi.

Una parte dell'esame prevede l'utilizzo di quarto per lo svolgimento degli esercizi.

A questo link trovate maggiori informazioni su quarto e una guida veloce sul suo funzionamento



(b) Selezione tipo di file/dati



In verde, lo YAML che controlla l'aspetto generale del documento

Modifica importante allo YAML

```

---
title: "Il mio file"
author: "Nome Cognome"
format:
  html:
    self-contained: true
editor: visual
---

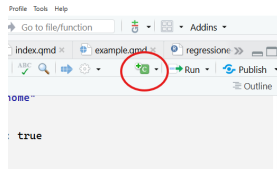
```

Il chunk di codice funziona esattamente come R – è sequenziale, va seguito l'ordine con cui vengono fatte le operazioni

Ogni volta che viene fatto il rendering del documento (`shift + ctrl + k` o freccina azzurra), viene creato un ambiente temporaneo dove viene eseguito il codice.

L'ambiente temporaneo è indipendente dall'ambiente che vedere su R!

Per creare un nuovo chunk: `shift + ctrl + i` oppure:



- 1 Rstudio
- 2 Working directories (wd)
- 3 Reporting con quarto
- 4 Elementi base**
- 5 Strutture
- 6 Funzioni utili
- 7 Monte Carlo
- 8 Esercizi liberi

- 1 Rstudio
- 2 Working directories (wd)
- 3 Reporting con quarto
- 4 Elementi base**
 - Calcolatrice
 - Assign
 - Nominare gli oggetti
- 5 Strutture
- 6 Funzioni utili

- 1 Rstudio
- 2 Working directories (wd)
- 3 Reporting con quarto
- 4 Elementi base**
 - Calcolatrice
 - Assign
 - Nominare gli oggetti
- 5 Strutture
- 6 Funzioni utili

Tutto quello che viene creato in R viene chiamato *oggetto* tramite l' "assegnazione" con specifici simboli, ovvero `<-` oppure `=`:

```
x <- 4 ; x
```

[1] 4

X = 5 ; X

[1] 5

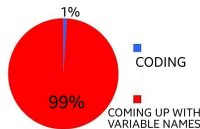
Gli elementi a destra del simbolo (contenuto) vengono assegnati all'elemento a sinistra (contenitore) del simbolo

Attenzione!

R è case sensitive. x e X sono due oggetti diversi!

- 1 Rstudio
- 2 Working directories (wd)
- 3 Reporting con quarto
- 4 Elementi base**
 - Calcolatrice
 - Assign
 - Nominare gli oggetti
- 5 Strutture
- 6 Funzioni utili

Nominare gli oggetti



Nomi validi sono lettere, numeri, punti, underscore (e.g., `variable_name`)

I nomi NON possono iniziare con numeri, simboli o valori interni di R

Buoni nomi per le variabili:

- `mean_rt`
- `sum_age`
- `prop_item`

Pessimi nomi per variabili:

- `x`
- `y`
- `ciccio`

Nominare gli oggetti

```
.01 = 5
```

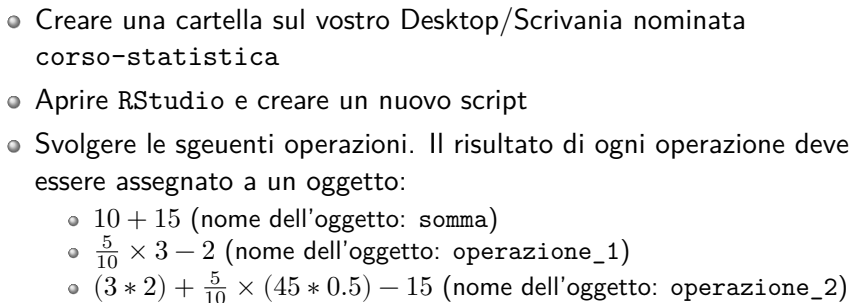
Error in 0.01 = 5: membro di sinistra dell'assegnazione (do_set) non valido

```
1 = "a"
```

Error in 1 = "a": membro di sinistra dell'assegnazione (do_set) non valido

```
TRUE = 567
```

Error in TRUE = 567: membro di sinistra dell'assegnazione (do_set) non valido



- 1 Rstudio
- 2 Working directories (wd)
- 3 Reporting con quarto
- 4 Elementi base
- 5 Strutture**
- 6 Funzioni utili
- 7 Monte Carlo
- 8 Esercizi liberi

- ① Rstudio
- ② Working directories (wd)
- ③ Reporting con quarto
- ④ Elementi base
- ⑤ Strutture
 - Pacchetti e Funzioni
 - Vettori
 - Indicizzare i vettori
 - Matrici
 - Indicizzare le matrici: In pratica


```
mean(x, trim = 0, na.rm = FALSE, ...)
```

- Nome della funzione: `mean`
- Parentesi che racchiudono gli argomenti della funzione
- argomenti:
 - obbligatori, senza valori predefiniti senza di cui la funzione non può funzionare
 - facoltativi con valori di default (`trim = 0, na.rm = FALSE`)

```
mean(1:5)
```

```
[1] 3
```

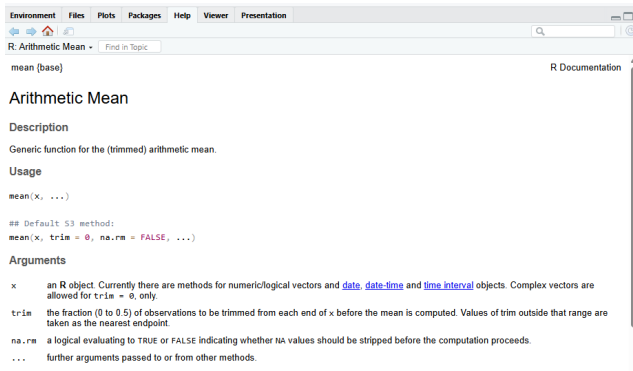
```
mean(1:5, trim = 0.3)
```

```
[1] 3
```


◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ≡ ≡ ↺ 🔍 ↻

?funzione permette di accedere all'help della funzione:

?mean



The screenshot shows the Rstudio Help window for the 'mean' function. The window has a menu bar with 'Environment', 'Files', 'Plots', 'Packages', 'Help', 'Viewer', and 'Presentation'. Below the menu bar is a search bar and a 'Find in Topic' button. The main content area displays the following information:

mean (base)

Arithmetic Mean

Description

Generic function for the (trimmed) arithmetic mean.

Usage

```
mean(x, ...)
```

Default S3 method:

```
mean(x, trim = 0, na.rm = FALSE, ...)
```

Arguments

- x** an R object. Currently there are methods for numeric/logical vectors and [date](#), [date-time](#) and [time interval](#) objects. Complex vectors are allowed for `trim = 0`, only.
- trim** the fraction (0 to 0.5) of observations to be trimmed from each end of `x` before the mean is computed. Values of trim outside that range are taken as the nearest endpoint.
- na.rm** a logical evaluating to TRUE or FALSE indicating whether NA values should be stripped before the computation proceeds.
- ...** further arguments passed to or from other methods.

Concatena diversi oggetti insieme per creare un nuovo oggetto unico

[1] 1 2 3

[1] 1 2 3

```
[1] TRUE TRUE TRUE
```

- ① Rstudio
- ② Working directories (wd)
- ③ Reporting con quarto
- ④ Elementi base
- ⑤ Strutture
 - Pacchetti e Funzioni
 - Vettori
 - Indicizzare i vettori
 - Matrici
 - Indicizzare le matrici: In pratica

Tipi di oggetti diversi → Tipi di vettori diversi:

- `num` (`numeric`): Numeri (-0.56, 3, 4.5)
- `logi` (`logic`): Logico (TRUE vs FALSE)
- `chr` (`character`): characters ("a", "b", "c")
- `factor` (`factor`): factor con diversi livelli

La distinzione più importanti è tra vettori di tipo categoriale (`chr` e `factor`) e vettori di tipo numerico (`int` e `num`):

- `integer` e `numeric`: Operazioni matematiche E calcolo delle frequenze
- `character` e `factor`: Calcolo delle frequenze

```
months = c(5, 6, 8, 10, 12, 16); months
```

[1] 5 6 8 10 12 16

```
weight = seq(3, 11, by = 1.5) ; weight
```

```
[1] 3.0 4.5 6.0 7.5 9.0 10.5
```

I vettori logici si ottengono testando delle condizioni sulla base dei valori di altri vettori:

months

[1] 5 6 8 10 12 16

Prendere solo i valori > 12

```
months > 12
```

```
[1] FALSE FALSE FALSE FALSE FALSE  TRUE
```

```
v_chr = c(letters[1:3], LETTERS[4:6]); v_chr
```

```
[1] "a" "b" "c" "D" "E" "F"
```

```
ses = factor(rep(c("low", "medium", "high"), each = 2)); ses
```

```
[1] low      low      medium medium high      high
Levels: high low medium
```


Attenzione a non mischiare gli oggetti!!

`num + logi → num`

`num + factor → num`

`num + chr → chr`

`chr + logi → chr`

- 1 Rstudio
- 2 Working directories (wd)
- 3 Reporting con quarto
- 4 Elementi base
- 5 **Strutture**
 - Pacchetti e Funzioni
 - Vettori
 - Indicizzare i vettori
 - Matrici
 - Indicizzare le matrici: In pratica

```
nomi = c("Pasquale", "Egidio", "Debora", "Luca", "Andrea")
nomi
```

```
[1] "Pasquale" "Egidio"    "Debora"    "Luca"      "Andrea"
```

Pasquale	Egidio	Debora	Luca	Andrea
----------	--------	--------	------	--------

1

2

3

4

5

```
nome_vettore[indice]
```

Pasquale	Egidio	Debora	Luca	Andrea
1	2	3	4	5

Pasquale	Egidio	Debora	Luca	Andrea
1	2	3	4	5

nomi[1] →


```
weight
```

```
[1] 3.0 4.5 6.0 7.5 9.0 10.5
```

```
weight[2] # secondo elemento di weight
```

```
[1] 4.5
```

```
(weight[6] = 15.2) # sostituisce il sesto elemento di weight
```

```
[1] 15.2
```

```
weight[seq(1, 6, by = 2)] # elementi 1, 3, 5
```

```
[1] 3 6 9
```

```
weight[2:6] # elementi dal 2 al 6 (incluso)
```

```
[1] 4.5 6.0 7.5 9.0 15.2
```

Si può indicizzare con la logica

```
weight > 7
```

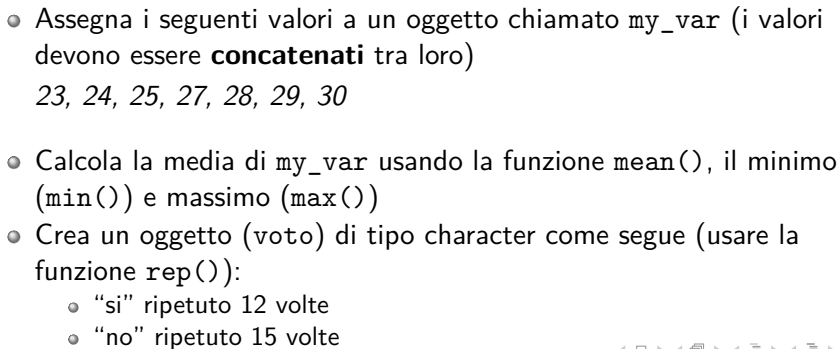
```
[1] FALSE FALSE FALSE  TRUE  TRUE  TRUE
```

```
weight[weight > 7] # solo i valori > di 7
```

[1] 7.5 9.0 15.2

```
weight[weight >= 4.5 & weight < 8] # valori compresi tra 4.5 e 8
```

[1] 4.5 6.0 7.5



- 1 Rstudio
- 2 Working directories (wd)
- 3 Reporting con quarto
- 4 Elementi base
- 5 **Strutture**
 - Pacchetti e Funzioni
 - Vettori
 - Indicizzare i vettori
 - Matrici
 - Indicizzare le matrici: In pratica

Matrici

```
`matrix(data, nrow, ncol, byrow = FALSE)`
```

```
# crea una matrice con 3 righe e 4 colonne  
# (i valori sono sistemati per riga)  
A = matrix(1:12, nrow=3, ncol = 4, byrow = TRUE)  
# assegna dei nomi alle righe  
rownames(A) = paste("a",  
                     1:nrow(A), sep = "_")  
# assegna dei nomi alle colonne  
colnames(A) = paste("b",  
                    1:ncol(A), sep = "_")
```

A

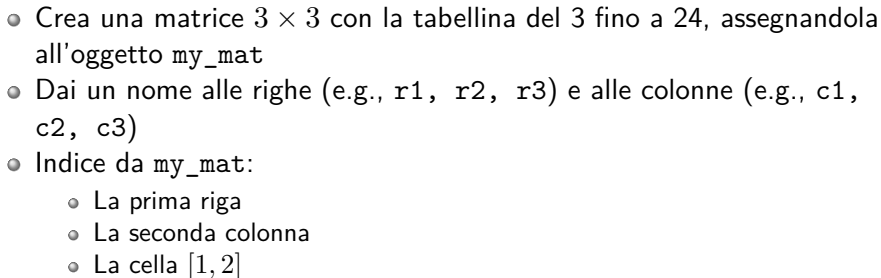
	b_1	b_2	b_3	b_4
a_1	1	2	3	4
a_2	5	6	7	8
a_3	9	10	11	12

- ① Rstudio
- ② Working directories (wd)
- ③ Reporting con quarto
- ④ Elementi base
- ⑤ Strutture
 - Pacchetti e Funzioni
 - Vettori
 - Indicizzare i vettori
 - Matrici
 - Indicizzare le matrici: In pratica

A

	b_1	b_2	b_3	b_4
a_1	1	2	3	4
a_2	5	6	7	8
a_3	9	10	11	12

$$A[1, \] \rightarrow 1, 2, 3, 4$$
$$A[2,] \rightarrow 5, 6, 7, 8$$
$$A[2, 3] \rightarrow 7$$



- 1 Rstudio
- 2 Working directories (wd)
- 3 Reporting con quarto
- 4 Elementi base
- 5 Strutture
- 6 Funzioni utili**
- 7 Monte Carlo
- 8 Esercizi liberi

- `mean()`: calcola la media di un vettore numerico
- `sd()`: calcola la deviazione standard di un vettore numerico
- `table()`: calcola le frequenze per i livelli di una variabile
- `sort()`: ordina una variabile in base ai valori (dal più piccolo al più grande)
- `sample()`: permuta i valori all'interno di un vettore, permette di creare vettori casuali di valori ecc

```
my_var = c(23, 24, 27, 28, 29, 30); my_var
```

```
[1] 23 24 27 28 29 30
```

```
voto = c(rep("si", 12), rep("no", 15)); voto
```

```
[1] "si" "si" "si" "si" "si" "si" "si" "si" "si" "si" "si" "si" "no" "no" "no"
[16] "no" "no" "no" "no" "no" "no" "no" "no" "no" "no" "no" "no" "no" "no" "no"
```

```
table(my_var)
```

```
my_var
```

23	24	27	28	29	30
1	1	1	1	1	1

```
table(voto)
```

voto	
no	si
15	12

```
sample(x, size, replace = FALSE, prob = NULL)
```

```
set.seed(2026) # ci assicura di osservare sempre lo stesso risultato
voto
```

```
[1] "si" "si" "si" "si" "si" "si" "si" "si" "si" "si" "si" "si" "no"
[16] "no" "no" "no" "no" "no" "no" "no" "no" "no" "no" "no" "no" "no"
```

```
sample(voto)
```

```
[1] "no" "si" "si" "no" "no" "si" "si" "no" "si" "no" "si" "si" "si"
[16] "no" "si" "no" "no" "si" "si" "no" "no" "no" "no" "no" "si" "no"
```

Estrarre un campione di ampiezza 10 dal vettore

```
sample(voto, size = 10)
```

```
[1] "si" "si" "no" "si" "si" "si" "no" "si" "no" "si"
```

Creare un nuovo vettore di lunghezza 10 dove il sì abbia una probabilità del 70% e il no una probabilità del 30%

```
new_voto = sample(c("si", "no"), size = 10, replace = TRUE,  
                  prob = c(.70, .30))  
new_voto
```

```
[1] "si" "no" "si" "no" "no" "si" "si" "no" "si" "si"
```

```
table(new_voto)/length(new_voto)
```

```
new_voto  
  no  si  
0.4 0.6
```

- 1 Rstudio
- 2 Working directories (wd)
- 3 Reporting con quarto
- 4 Elementi base
- 5 Strutture
- 6 Funzioni utili**
 - Basi di programmazione: Il ciclo for
- 7 Monte Carlo

Serve per iterare un numero predefinito di volte la stessa operazione (loop)

```
for (variable in vector) {
  computation
}
```

variable: un indice che si muove attraverso vector

vector: definisce il numero di iterazioni

computation: operazioni che vengono svolte all'interno del ciclo for

```
replicate = 100
results = numeric(100)
set.seed(123)
for (i in 1:replicate) {
  temp_variable = rexp(200, rate = i)
  results[i] = mean(temp_variable)
}
length(results)
```

①
②
③
④
⑤

- ① Definisce il numero di iterazioni
- ② Inizializza un vettore di lunghezza 100 per salvare i risultati
- ③ “Pianta il seme” per rendere i risultati replicabili
- ④ La variabile *i* si muove attraverso i valori che vanno da 1 a 100 (1:replicate)
- ⑤ Qui si svolge realmente l'operazione di generare i numeri casuali da una distribuzione esponenziale dove il *rate* che ne definisce la velocità di accelerazione è definito da *i* mentre alla riga successiva si calcola la media per ogni variabile e la si assegna all'*i*-esima posizione del vettore *results*

[1] 100

- 1 Rstudio
- 2 Working directories (wd)
- 3 Reporting con quarto
- 4 Elementi base
- 5 Strutture
- 6 Funzioni utili
- 7 Monte Carlo**
- 8 Esercizi liberi

Questo permette di stimare quanto è verosimile un fenomeno dati i parametri iniziali

- 1 Definire lo scenario (ad esempio, il lancio di una moneta)
- 2 Definire il numero di replicazioni che si vogliono implementare (dipende anche dalla potenza del PC)
- 3 Ripetere lo scenario (1.) per il numero di repliche (2.) e annotare il risultato osservato (Testa vs. Croce)
- 4 Calcolare varie statistiche descrittive rispetto ai risultati al (3.)

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

- 1 Rstudio
- 2 Working directories (wd)
- 3 Reporting con quarto
- 4 Elementi base
- 5 Strutture
- 6 Funzioni utili
- 7 Monte Carlo**
 - Esempio: La moneta è truccata?

Esempio: La moneta è truccata?

1. Lo scenario

```
moneta = c("testa", "croce")
```

2. Numero di repliche

```
repliche = 1000
```

```
chr [1:1000] "croce" "testa" "testa" "croce" "testa" "testa" "
```

Esempio: La moneta è truccata?

4. Calcolare le statistiche di interesse

```
table(mc) # frequenze
```

```
mc
croce testa
  480   520
```

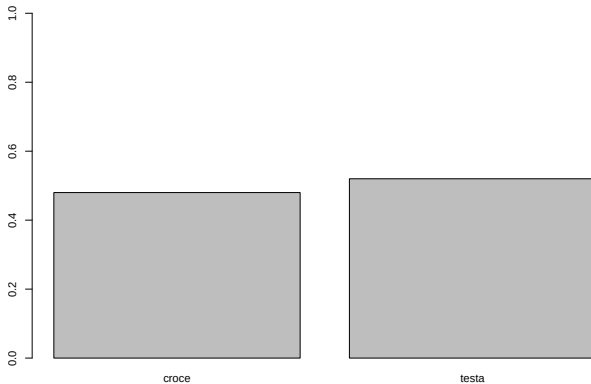
```
prop.table(table(mc)) # frequenze relative
```

```
mc
croce testa
  0.48  0.52
```


Esempio: La moneta è truccata?

4. Calcolare le statistiche di interesse

```
barplot(prop.table(table(mc)), ylim=c(0,1))
```



Esempio: La moneta è truccata?

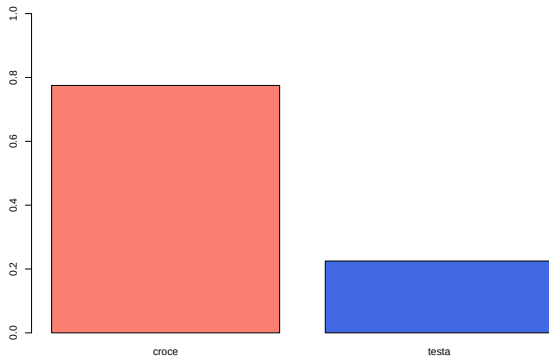


Figure 6: Questa moneta è truccata? Provate a ricostruire il codice!

P.S. `:set.seed(1903)`

Esempio: La moneta è truccata?

Soluzione

```
newmoneta = sample(moneta, repliche,
                    replace = TRUE, prob = c(.20, .80))
barplot(prop.table(table(newmoneta)),
        ylim= c(0,1), col = c("salmon", "royalblue"))
```

- 1 Rstudio
- 2 Working directories (wd)
- 3 Reporting con quarto
- 4 Elementi base
- 5 Strutture
- 6 Funzioni utili
- 7 Monte Carlo
- 8 Esercizi liberi**

- 1 Rstudio
- 2 Working directories (wd)
- 3 Reporting con quarto
- 4 Elementi base
- 5 Strutture
- 6 Funzioni utili
- 7 Monte Carlo
- 8 Esercizi liberi**

Trovare lo stesso risultato con una simulazione di Monte Carlo

```
mazzo = c("fiori", "cuori", "picche", "denari")
mazzo = paste0(mazzo, rep(1:13, each = 4))
```

Soluzione

```
mazzo = c("fiori", "cuori", "picche", "denari")
mazzo = paste0(mazzo, rep(1:13, each = 4))
set.seed(123)
repliche = 1000
results = numeric(repliche)
for (i in 1:repliche) {
  results[i] = sample(mazzo, 1)
}
table(results)
sum(grepl("fiori", results))/repliche
```

- 1 Rstudio
- 2 Working directories (wd)
- 3 Reporting con quarto
- 4 Elementi base
- 5 Strutture
- 6 Funzioni utili
- 7 Monte Carlo
- 8 Esercizi liberi**

Trovare lo stesso risultato con una simulazione di Monte Carlo

Codice per creare il mazzo

```
mazzo = c("fiori", "cuori", "picche", "denari")
mazzo = paste(mazzo, rep(c("asso", 2:13), each = 4), sep = "_")
```

Soluzione

```
mazzo = c("fiori", "cuori", "picche", "denari")
mazzo = paste(mazzo, rep(c("asso", 2:13), each = 4), sep = "_")
set.seed(123)
repliche = 1000
results = numeric(repliche)
for (i in 1:repliche) {
  results[i] = sample(mazzo, 1)
}
table(results)
sum(grepl("asso", results))/repliche
```

- 1 Rstudio
- 2 Working directories (wd)
- 3 Reporting con quarto
- 4 Elementi base
- 5 Strutture
- 6 Funzioni utili
- 7 Monte Carlo
- 8 Esercizi liberi**

- con reimmissione (eventi indipendenti)
- senza reimmissione (eventi dipendenti)

$$P(A) = \frac{13^2}{52^2} = \frac{1}{16} = 0.0625$$
$$P(A) = \frac{13^2 - 13}{52^2 - 52} = \frac{3}{51} = 0.05882353$$

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ≡ ↺ 🔍 ↻

- 1 Rstudio
- 2 Working directories (wd)
- 3 Reporting con quarto
- 4 Elementi base
- 5 Strutture
- 6 Funzioni utili
- 7 Monte Carlo
- 8 Esercizi liberi**

Soluzione

```
moneta = c("T", "C")
repliche = 10000
results = numeric(repliche)

for (i in 1:repliche) {
  out = sample(moneta, 10, replace = TRUE)
  results[i] = sum(out == "C") >= 6
}

sum(results)
```

[1] 3867

```
sum(results)/repliche
```

```
[1] 0.3867
```

- 1 Rstudio
- 2 Working directories (wd)
- 3 Reporting con quarto
- 4 Elementi base
- 5 Strutture
- 6 Funzioni utili
- 7 Monte Carlo
- 8 Esercizi liberi**

Si hanno due urne definite come segue:

Urna 1

Una pallina verde (V)

Una pallina blu (B)

Urna 2

Una pallina verde (V)

Una pallina rossa (R)

Calcolare la probabilità che estraendo una pallina da ogni urna, le due palline siano dello stesso colore

Ne segue che:

$$P(A) = \frac{|A|}{|\Omega|} = \frac{1}{4} = 0.25$$

MC? (`set.seed(123)`)

Soluzione

```
set.seed(123)
u1 = c("verde", "blu")
u2 = c("verde", "rossa")
repliche = 1000
results = numeric(repliche)
for (i in 1:repliche) {
  out1 = sample(u1,1)
  out2 = sample(u2, 1)
  results[i] = all(out1 == "verde", out2 == "verde")
}
table(results)/repliche
```

```
results
      0      1
0.736 0.264
```

- 1 Rstudio
- 2 Working directories (wd)
- 3 Reporting con quarto
- 4 Elementi base
- 5 Strutture
- 6 Funzioni utili
- 7 Monte Carlo
- 8 Esercizi liberi**

B = La seconda pallina estratta è rossa

```
urna = c(rep("rossa", 4), rep("blu", 3))
```

